`test_smpl/` — the three shared test runners

> **The test applications are no longer here.** Every board owns its own, in `test/<board>/` of the architecture project that has that board. How they are laid out and how to run them is tests-in-aarch32-aarch64.md.
>
> This directory kept only what is genuinely shared and genuinely not a test: the three scripts that drive a build.

```
test_smpl/
├── run-qemu.sh      run a build's *-qemu images under an emulator
├── run-host.sh      run a build's *-host executables (the POSIX port)
├── run-hw.sh        run one *-hwd image on silicon, through OpenOCD
├── include/hw_result.hpp       the verdict contract every board shares
├── src/board-contract.cpp      its default implementation (the kernel's
│                               micro-os-plus::test-support target)
└── no-power-cycle/             OpenOCD configs and run-hw-simple.sh for
                                boards that need no power cycle
```

On the rebuilt `smp` branch these arrive as four commits (xpack-dev-smp.md K43a–K43d): the verdict contract, then each runner. `test_smpl/` stays a real directory at the kernel root, referenced by the platforms as `../test_smpl`.

No script knows a board or a port. They are called two ways: a board's `test/boards/<id>/{qemu,hw,run}.sh` supplies the facts and execs one of them, and the xPack harness's port platforms (`tests/platforms/{aarch32,aarch64}-*/CMakeLists.txt`, `native`) call `run-qemu.sh` and `run-host.sh` directly from their CTest cases, passing the same facts as arguments and environment.

`run-qemu.sh <build-test-dir> <qemu-binary> <machine-args…>`

Runs every `*-qemu.bin` in the directory and reports `PASS` / `SKIP` / `FAIL` from the `RESULT:` line each test prints, then a one-line summary.

variable	meaning
`UOS_QEMU_ONLY`	run just this application, with its output live on the terminal instead of only in a log — the run you are watching during bring-up
`UOS_TEST_SRC_DIR`	the board's `test/` directory, so a test's host-side helper can be found (`sd_test/flatfs_tool.py`, which seeds the card image)
`UOS_QEMU_SHIM`, `UOS_QEMU_LOAD_ADDR`	for a port that cannot be handed straight to `-kernel`: the AArch32 BCM2837 build, because QEMU's `raspi3b` starts its Cortex-A53 cores in AArch64, so a 20-line stub drops to AArch32 and jumps to the image

Tests that talk to an SD card get one: `sd_test` a seeded flatfs volume, the others a 4 GiB sparse blank. Both are kept out of git.

A suite is read from its summary, so each run is captured to `.qemu-logs/<app>.log`. `UOS_QEMU_ONLY` streams as well as captures.

`run-host.sh <build-test-dir>`

The sibling of `run-qemu.sh` with the emulator taken out: it runs every `*-host` executable in the directory, with the same timeout table, the same `PASS` / `SKIP` / `FAIL` / `TIMEOUT` / `NO-RESULT` verdicts, the same summary line and the same exit status. Logs go to `.host-logs/<app>.log`.

variable	meaning
`UOS_RUN_ONLY`	run just this test, echoed live as well as logged

`run-hw.sh <build-test-dir> <app|list> [run-seconds]`

Halts the cores, enables semihosting, `load_image`, resumes — and watches the semihosted console for the verdict.

variable	meaning
`UOS_HW_CFG`	the board's OpenOCD config
`UOS_HW_CFG_INIT`	`1` when that config is purely declarative and the runner must issue `init`
`UOS_HW_ENTRY`	where the image is linked (read from the ELF when possible)
`UOS_HW_SPIN_WORDS`	`__smp_spin` words to zero before releasing the secondaries; `0` skips that stage
`UOS_HW_RESUME`	`cpsr` | `pc` | `entry` (every core) — or `cpsr-first` | `pc-first`: only the first core is started at the entry, the others are resumed where they were halted (the Pi's firmware loop) until the kernel releases them; used with `UOS_HW_SPIN_WORDS=0` by both Pi ports
`UOS_HW_NCPU`, `UOS_HW_CORES`, `UOS_HW_TARGET_FMT`	how many cores, which are debug targets at load time, and how they are named (`bcm2837.cpu%d`, `rk3506.a7.%d`)
`UOS_HW_PRELOAD`	Tcl to run between halt and load — the Lyra's MMU/cache sanitize, which has to happen before an image is written over the miniloader's page tables
`UOS_HW_NM`, `UOS_HW_READELF`, `UOS_HW_ADAPTER_KHZ`	binutils for the ELF, and the probe clock

Pure OpenOCD. No GDB — the board configs set `gdb port disabled` outright.

No redirection. Everything OpenOCD and the board write reaches your terminal as it happens; the tee'd copy in `.hw-logs/<app>.log` exists only so the loop can match `RESULT:`. The script never opens the tty, so your own `tio -b 115200 /dev/ttyACM0` is never fought over.

It refuses a suite. Every run is a `load_image` into RAM over whatever the previous test left there, and neither board has a reset a script can drive. It takes exactly one test; you power-cycle between them. `list` is the default argument and prints what a build has, with each budget.

`run-qemu.sh` and `run-hw.sh` replace what the predecessor repository kept as `hw.sh` + `hw-olimex.sh` in every test directory of every port — about 200 near-identical lines, 48 files. The probe choice that `hw-olimex.sh` made is now `PROBE=jlink|olimex` on the Pi boards' `hw.sh` (and, on the Zero 2 W / Pi 3 B of both ports, `PROBE=cmsis-dap`, `openocd-cmsisdap-rpi3.cfg`).